NAMA: REYHAN GILANG RAMADHAN

KELAS: SI-IIIA

Soal 1 — Konsep Abstraction

Jawaban:

Abstraction adalah konsep dalam OOP yang digunakan untuk menyembunyikan detail proses atau cara kerja di dalam program dan hanya menampilkan bagian penting yang perlu digunakan oleh pengguna.

Contohnya adalah mesin ATM. Pengguna hanya perlu memasukkan kartu, PIN, dan memilih menu penarikan uang tanpa harus mengetahui bagaimana sistem ATM memproses transaksi di dalamnya.

Jadi, abstraction membantu membuat program lebih sederhana karena pengguna cukup mengetahui fungsi yang tersedia tanpa harus memahami seluruh proses di baliknya.

Soal 2 — Abstract Class dan Abstract Method

Jawaban:

Abstract class adalah class khusus yang digunakan sebagai rancangan atau dasar bagi class lain. Class ini biasanya berisi atribut dan method yang harus dimiliki oleh class turunannya.

Sedangkan abstract method adalah method yang hanya memiliki deklarasi tanpa isi proses yang lengkap dan harus dibuat ulang oleh subclass.

Abstract class tidak dapat diinstansiasi secara langsung karena class tersebut belum memiliki implementasi lengkap dari abstract method yang diwajibkan.

Dengan demikian, subclass harus melengkapi method yang sudah ditentukan sebelum object dapat dibuat.

Soal 3 — Prediksi Output

Jawaban:

Output yang muncul ketika program dijalankan adalah:

```
Mahasiswa
```

Penjelasan:

Output tersebut muncul karena class Mahasiswa merupakan turunan dari abstract class Pengguna dan sudah memiliki implementasi method `tampilkan_peran()`.

Decorator `@abstractmethod` digunakan untuk menandai sebuah method sebagai method abstrak yang wajib dibuat atau diimplementasikan oleh subclass.

Pada contoh tersebut, class Mahasiswa sudah mengimplementasikan method `tampilkan_peran()` sehingga object Mahasiswa dapat dibuat dan program berjalan dengan baik.

Soal 4 — Error pada Abstract Class

Jawaban:

Ketika perintah `Mahasiswa()` dijalankan, program akan menghasilkan error karena class Mahasiswa belum mengimplementasikan abstract method `tampilkan_peran()` yang diwarisi dari class Pengguna.

Pesan error yang muncul kurang lebih:

```
TypeError: Can't instantiate abstract class Mahasiswa
without an implementation for abstract method
'tampilkan_peran'
```

Penjelasan:

Class Pengguna memiliki abstract method `tampilkan_peran()` yang wajib dibuat ulang oleh setiap subclass.

Namun, pada contoh tersebut class Mahasiswa hanya mewarisi class Pengguna tanpa membuat implementasi method tersebut.

Akibatnya, Python tidak mengizinkan object Mahasiswa dibuat karena masih ada method abstrak yang belum dilengkapi.

Soal 5 — Merancang Abstract Class

Jawaban:

Berikut contoh abstract class `Transaksi` dengan abstract method `proses()`, kemudian dibuat subclass `TransaksiTunai`yang mengimplementasikan method tersebut.

Kode Python:

```
from abc import ABC, abstractmethodclass Transaksi(ABC):
@abstractmethod    def proses(self):        passclass
TransaksiTunai(Transaksi):    def proses(self):
print("Transaksi tunai berhasil diproses")transaksi =
TransaksiTunai()transaksi.proses()
```

Output:

```
Transaksi tunai berhasil diproses
```

Penjelasan:

- `Transaksi` merupakan abstract class yang menjadi dasar untuk membuat jenis transaksi.
- `proses()` merupakan abstract method yang wajib diimplementasikan oleh subclass.
- `TransaksiTunai` merupakan subclass yang memberikan isi atau proses pada method `proses()`.
- `transaksi.proses()` digunakan untuk menjalankan proses transaksi tunai.

Manfaat kontrak perilaku:

Kontrak perilaku berfungsi untuk memastikan setiap subclass memiliki method yang sudah ditentukan oleh abstract class.

Dengan adanya aturan tersebut, setiap jenis transaksi harus memiliki method `proses()` meskipun cara kerjanya berbeda-beda.

Hal ini membuat program lebih teratur, mudah dikembangkan, dan mengurangi kemungkinan adanya method penting yang belum dibuat.

Kesimpulan

Abstraction merupakan konsep OOP yang berfungsi menyembunyikan detail implementasi dan hanya menampilkan fungsi yang diperlukan. Dengan menggunakan abstract class dan abstract method, programmer dapat menentukan aturan dasar yang harus diikuti oleh class turunannya sehingga program menjadi lebih terstruktur, konsisten, dan mudah dikembangkan.